

CO_Homework 6_Verilog 2_2026/05/27

班級：電資二 (北聯/北醫)

學號：B812111001

姓名：張博翔

組別：Group 7、成員：113820028 張宏奕、114054812 陳半半(隨班附讀)

討論時間：2026/05/27 (三) 23:00

方式：線上討論

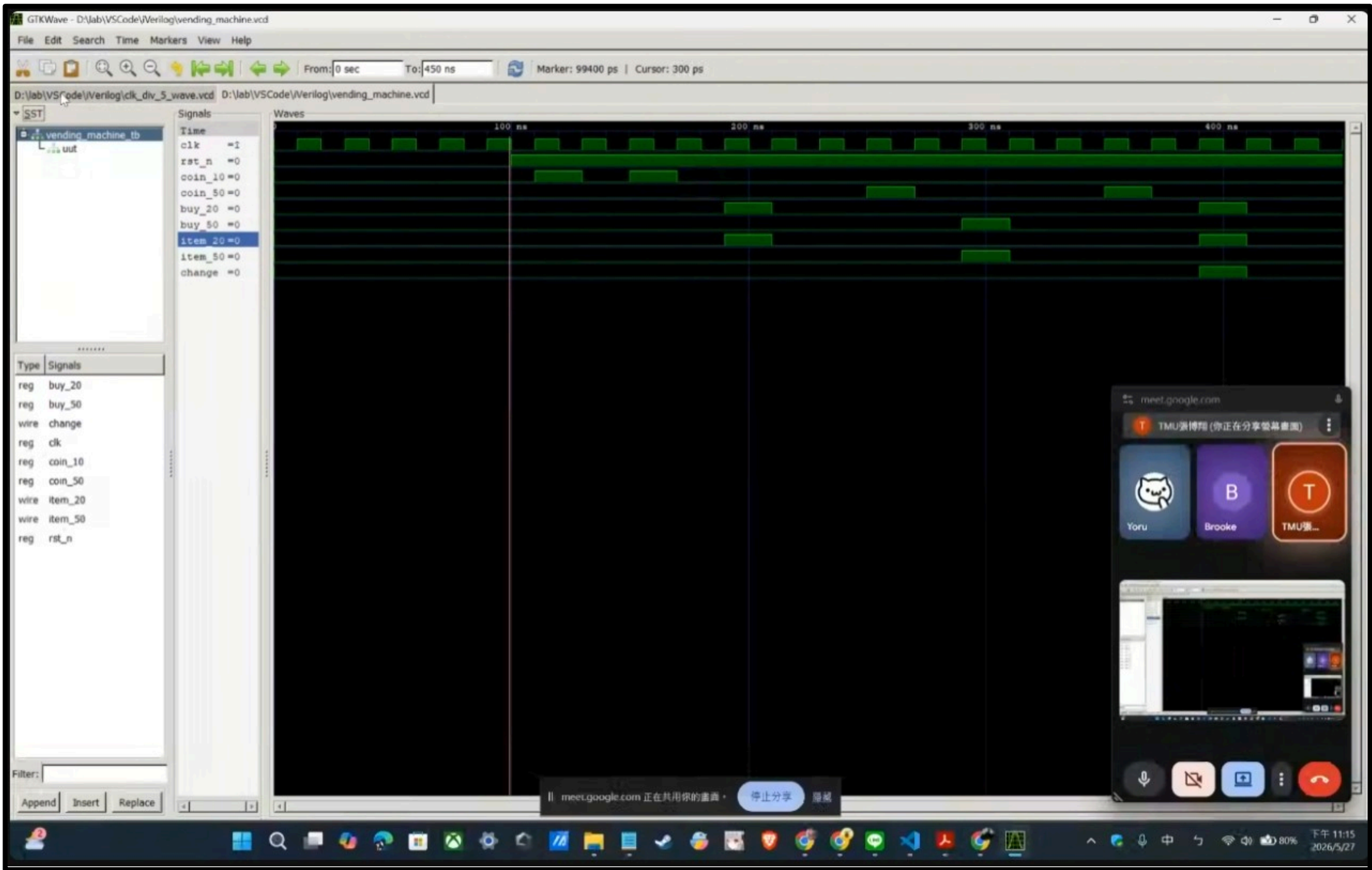
討論內容與心得：

有限狀態機 (FSM) 的精確控制與防錯機制：在實作自動販賣機的米利狀態機 (Mealy Machine) 時，我們深刻體會到將「狀態暫存 (Sequential)」與「次態/輸出邏輯 (Combinational)」分離的「三段式狀態機」架構的重要性。透過這次實驗，我們不僅學會了如何處理投幣與找零的複雜狀態轉移，更解決了軟體思維轉硬體思維時常犯的錯誤（例如將輸入訊號與時脈寫在同一個 block 導致的延遲）。同時，我們也在 Testbench 驗證中發現並解決了因訊號與時脈邊緣重合所引發的競爭危害 (Race Condition)，大幅提升了電路模擬的穩定度。

ALU 功能擴充與語法對硬體合成的影響：本次 8-bit ALU 的設計中，我們為了涵蓋高達 13 種的算術與邏輯功能，首先將指令碼 (Opcode) 由原本的 3-bit 擴展至 4-bit。最值得探討的是，作業限制我們必須完全使用 if-else 語法來取代原本的 case。在討論與驗證的過程中，我們意識到這不僅僅是語法上的轉換，更會直接影響底層的硬體架構：連續的 if-else 會被邏輯合成器 (Synthesizer) 推斷為具有「優先權 (Priority)」的階層式多工器鏈。這讓我們親身評估並了解到，排在末端的指令（如位移運算）會因此面臨較長的關鍵路徑延遲 (Critical Path Delay)，進而影響整體 ALU 的最高工作頻率。

奇數除頻器 (Divide by 5) 的邏輯推演與實作：基於講義中除 3 除頻器的基礎，我們進一步挑戰了非整數的 Divide by 5 除頻器。為了解決計數上限的問題，我們將原本的暫存器擴充為 3-bit（以容納 0~4 的狀態計數）。在實作過程中，我們仔細推演了如何在特定計數值搭配時脈的正、負沿觸發 (posedge / negedge) 來翻轉輸出訊號，以盡可能逼近理想的工作週期 (Duty Cycle)。這次修改不僅鞏固了我們對計數器 (Counter) 的掌控，也讓我們對時脈域 (Clock Domain) 的精準切割有了更深一層的實務經驗。

會議照片：



Homework Q1 : Divide by 5 Counter

Verilog HDL :

Icarus Verilog & GTK wave (vscode) :

```
1.  //=====
    =====
2.  //Homework1:Divide by5 Counter (50% DutyCycle)
3.  //=====
    =====
4.  module ClockBy5(clk,reset,clk_out);
5.      // 宣告
6.      input clk;      // 原始時脈
7.      input reset;    // 同步重置訊號 (高電位有效)
8.      output clk_out; // 除以 5 且具備 50%duty 的輸出時脈
9.
10.     // 使用 3-bitreg 進行計數 (範圍 0 到 4)
11.     reg [2:0] pos_count,neg_count;
12.
13.     //Continuous Assignment (組合邏輯)
14.     // 當正沿或負沿計數器的數值 >= 3 亦即 3 或 4 時，輸出時脈為高電位
15.     // 兩者錯開半拍疊加，剛好產生 2.5 個週期的高電位與 2.5 個週期的低電位
16.     assign clk_out = ((pos_count >= 3) | (neg_count >= 3));
17.
18.     // 正沿計數器：0->1->2->3->4->0 循環
19.     always@(posedge clk) begin
20.         if(reset)
21.             pos_count <= 3'd0;          // 重置時歸零
22.         elseif(pos_count == 3'd4)
23.             pos_count <= 3'd0;          // 數到 4 時歸零重新計數 (共 5 個狀態)
24.         else
25.             pos_count <= pos_count + 1'b1; // 其餘情況遞增 1
26.     end
27.
28.     // 負沿計數器：0->1->2->3->4->0 循環
29.     always@(negedge clk) begin
30.         if(reset)
31.             neg_count <= 3'd0;          // 重置時歸零
32.         elseif(neg_count == 3'd4)
```

```

33.         neg_count <= 3'd0;           // 數到 4 時歸零重新計數
34.     else
35.         neg_count <= neg_count + 1'b1; // 其餘情況遞增 1
36.     end
37. Endmodule
38.

```

testbench :

```

1.  `timescale 1ns/100ps  // 設定時間單位為 1ns , 時間精度為 100ps
2.  `include "clk_div.v"  // 導入 Divideby5Counter 模組檔名
3.
4.  module clkdiv5_tb;
5.      reg clk, reset;
6.      wire clk_out;
7.
8.      // Instantiate the Module
9.      // 將定義好的 ClockBy5 電路實例化為 INST
10.     ClockBy5 INST (
11.         .clk(clk),
12.         .reset(reset),
13.         .clk_out(clk_out)
14.     );
15.
16.     // 時脈產生電路
17.     initial clk = 1'b0;      // 初始化時脈訊號為 0
18.     always #5 clk = ~clk;    // 每 5ns 反相一次 , 產生一個週期為 10ns (頻率 100MHz)
    的輸入時脈
19.
20.     // 測試
21.     initial begin
22.         // 初始化 reset
23.         reset = 1'b0;
24.
25.         // 初始化 clk_out 的內部計數器暫存器
26.         INST.pos_count = 3'b000;
27.         INST.neg_count = 3'b000;
28.
29.         #5 reset = 1'b1;     // 模擬開始 5ns 後 , 拉高重置訊號 , 初始化內部電路狀態

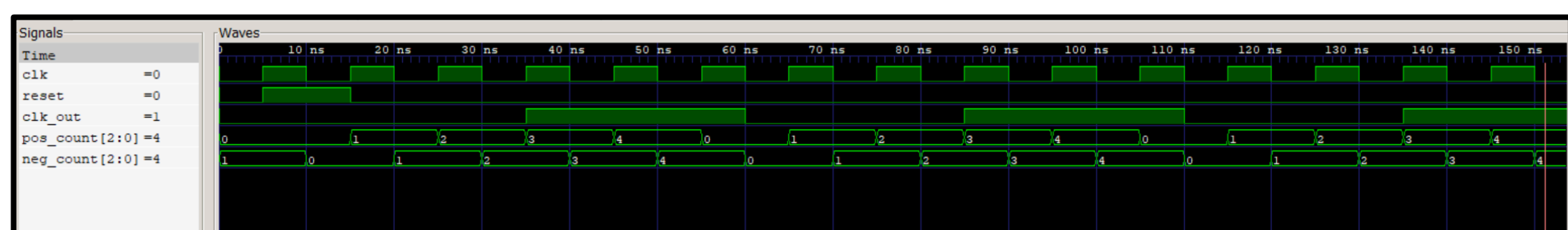
```

```

30.         #10reset = 1'b0;    // 再過 10ns 後，放開重置訊號，讓電路開始正常除 5 運
    作
31.         #150 $finish;      // 當模擬時間達到 150ns 時，自動呼叫系統任務結束模擬，
    防止無限模擬崩潰
32.     end
33.
34.     // 系統監視文字輸出 (SystemMonitor)
35.     initialbegin
36.         // 每當 clk、reset 或 clk_out 的狀態發生跳變時，在終端機列印數值
37.         $monitor("Time=%0d ns,clk=%b,reset=%b,clk_out=%b",$time,clk,reset,
    clk_out);
38.     end
39.
40.     //VCDWaveformDump
41.     initialbegin
42.         $dumpfile("clk_div_5_wave.vcd");
43.         $dumpvars(0,clkdiv5_tb);
44.     end
45. Endmodule
46.

```

Simulation Result：



說明：

設計核心

在數位邏輯中，除以偶數（例如除以 4）只需要讓計數器數 2 個週期高電位、2 個週期低電位即可達成 50% 的對稱比例。

除以 5，若單純只使用時脈的正沿觸發（posedge），只能選擇「3 個週期高、2 個週期低」或「2 個週期高、3 個週期低」，會導致波型不對稱（DutyCycle 為 60% 或 40%）。

為了得到精準的 50%（也就是高低電位各佔 2.5 個週期），設計上必須同時借用「正沿」與「負沿」來獲取那 0.5 拍的解析度。

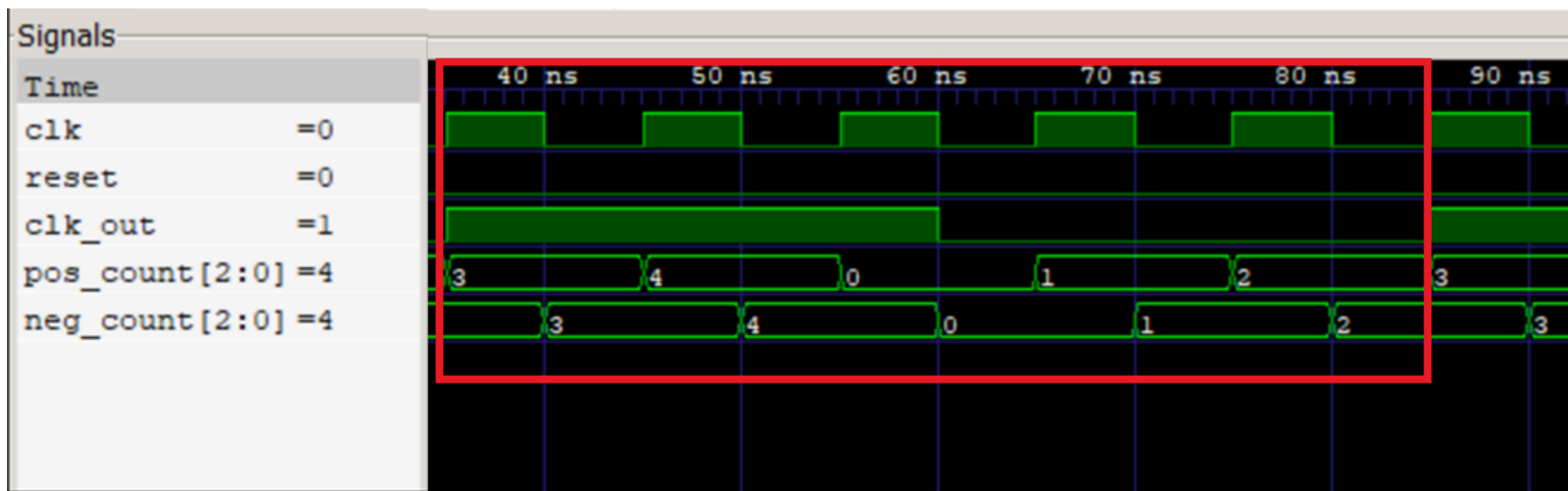
程式邏輯概念

正沿計數器（pos_count）：只在時脈往上打（posedge）時作動，從 0 數到 4 循環。

負沿計數器 (neg_count)：只在時脈往下掉 (negedge) 時作動，同樣從 0 數到 4 循環。因為是負沿觸發，它的數值變化永遠會比 pos_count 慢半個時脈週期 (0.5 拍)。

組合邏輯疊加 (assign)：利用位元或閘 (OR gate) 將兩者結合：assign clk_out = ((pos_count >= 3) | (neg_count >= 3));。只要其中一個計數器數到 3 或 4，輸出就會被強制拉成高電位 1。

波型(35ns~85ns 區間為例)



高電位開始 (35ns)：此時 clk 發生正緣，pos_count 率先進位變成 **3**。因為滿足了 (pos_count >= 3) 的條件，clk_out 瞬間被拉高為 1。

狀態交接點 (55ns)：clk 再次發生正緣，pos_count 數滿歸零變回 0。照理說條件失效 clk_out 應該要掉下來。但是，慢半拍的 neg_count 此時的數值是 4。透過 OR 閘邏輯，neg_count 成功把 clk_out 繼續在高電位。

高電位結束 (60ns)：經過了半個週期 (5ns)，clk 發生負緣，neg_count 終於也數滿歸零變回 0。此時兩個計數器都小於 3，所有條件失效，clk_out 正式落下變成 0。

從 35ns 升起到 60ns 落下，clk_out 維持高電位的時間是 25ns。接著從 60ns 到 85ns 維持低電位也是 **25ns**。總週期為 50ns，剛好是原始時脈 (10ns) 的 5 倍，高低電位對稱，達成 50% Duty Cycle 的目標。

Divideby5 Counter(奇數除頻器) 相對於 除 2 除 3 除頻器 應用的領域差異:

除 2 除頻器 (Even): 基礎除頻

主時脈樹 (ClockTree) 降頻：在處理器或大型 SoC 內部，通常由一個鎖相迴路 (PLL) 產生極高頻的時脈，然後一路透過除 2 的網路，分發給不同層級的子系統。

記憶體介面：例如 DDR (DoubleDataRate) 記憶體技術，其底層邏輯與時脈的倍頻/除 2 息息相關，用以同步資料的傳輸。

動態電壓與時脈調整 (DVFS)：當系統閒置時，最快速、最安全的降功耗方式，就是直接將時脈「砍半」(除 2 或除 4)，運用在作業系統底層的電源管理。

除 3/ 除 5 除頻器 (Odd): 頻率精確匹配

通訊協定與鮑率產生 (Baud Rate Generation)：標準通訊介面 (如 UART、SPI、I2C) 或網路協定，有其嚴格規定的工作頻率。假設系統主頻率為 100MHz，但需要一個 20MHz 的 SPI 介面來連接外部感測器，這時就必須精準地切出一個「除 5」的時脈域。

影音同步訊號：影像顯示 (如 VGA/HDMI 的 Pixel Clock) 與音訊處理 (I2S) 的取樣頻率通常不是 2 的冪次方。為了讓影像不撕裂、音訊不失真，常需要除 3、除 5 甚至非整數的分頻來對齊人類感官的標準更新率。

異質運算架構與 AI 加速器：在現代的 AI 基礎設施或大型分散式運算晶片中，系統極為龐大 (屬於 Globally Asynchronous Locally Synchronous, GALS 架構)。運算核心 (如 Tensor Cores) 可能跑在最高頻率，但負責資料搬運的匯流排 (Bus) 或網路互連介面 (Interconnect)，礙於物理限制只能跑在較低頻率。為了在同一個 PLL 下榨出最大的資料吞吐量，架構師會利用除 3、除 5 這種奇數除頻，調配出 333MHz 或 200MHz 的特製時脈，讓不同的硬體模組能在效能與功耗間取得最佳平衡。

Homework Q2：自動販賣機米利狀態機（Mealy FSM）

Verilog HDL：

Icarus Verilog & GTK wave (vscode)：

```
1.  //=====
   =====
2.  //Homework2: 自動販賣機米利狀態機 (MealyFSM)
3.  // 設計功能：模擬投入 10/50 元，並依據餘額決定是否吐出 20/50 元商品與找零
4.  //=====
   =====
5.
6.  module vending_machine(
7.      //Input Ports
8.      input clk,          // 系統時脈 (Clock)
9.      input reset,        // 非同步重置訊號 (Active-High)
10.
11.     input coin_10,       // 投幣輸入：投入 10 元硬幣
12.     input coin_50,       // 投入 50 元
13.     input buy_20,        // 按鈕輸入：購買 20 元商品
14.     input buy_50,        // 購買 50 元商品
15.
16.     //Output Ports
17.     output reg item_20,   // 動作輸出：吐出 20 元商品 (1 代表執行)
18.     output reg item_50,   // 動作輸出：吐出 50 元商品 (1 代表執行)
19.     output reg change     // 動作輸出：找零錢訊號 (1 代表執行找零)
20. );
21.
22.  //
   =====
   =
23.  //State assignment (使用 2 bits，對應 4 個狀態)
24.  //
   =====
   =
25.  parameter S_START    = 2'b00; // 狀態 00：初始狀態 (目前累計餘額為 0 元)
26.  parameter S_LESS     = 2'b01; // 狀態 01：錢貨不足 (目前累計餘額為 10 元)
27.  parameter S_CAN_20   = 2'b10; // 狀態 10：可買 20 元 (目前累計餘額為 20 元)
28.  parameter S_CAN_ALL  = 2'b11; // 狀態 11：可買所有 (目前累計餘額為 50 元以上)
```

```

29.
30.    // 宣告 current_state 與 next_state 的內部暫存器
31.    reg[1:0] current_state, next_state;
32.
33.    //
=====
=
34.    // 【Sequential Logic】
35.    // 更新狀態。
36.    // 當 reset 變為 1 強制歸零。
37.    //
=====
=
38.    always@(posedge clk or posedge reset) begin
39.        if(reset)
40.            current_state <= S_START;    // 高電位重置觸發：回到初始
41.        else
42.            current_state <= next_state; // 一般情況，時脈正緣更新至下一狀態
43.    end
44.
45.    //
=====
=
46.    // 【Combinational Logic】
47.    // 根據「目前狀態」與「外部投幣輸入」，即時算出「下一狀態」(Mealy)
48.    //
=====
=
49.    always@(*) begin
50.        next_state = current_state;    // 預設維持原狀態，防止合成出非預期 Latch
51.
52.        case(current_state)
53.            // 情況 00：目前餘額 0 元
54.            S_START: begin
55.                if(coin_10)    next_state = S_LESS;    // 投 10 元 -> 餘額 10 元
                    -> 跳到「錢不足」
56.                elseif(coin_50) next_state = S_CAN_ALL; // 直接投 50 元 -> 餘額
                    50 元 -> 跳到「可買所有」

```

```

57.         end
58.
59.         // 情況 01：目前餘額 10 元
60.         S_LESS:begin
61.             if(coin_10)    next_state=S_CAN_20;    // 再投 10 元 -> 累計 20
元 -> 跳到「可買 20 元」
62.             elseif(coin_50)next_state=S_CAN_ALL;    // 再投 50 元 -> 累計 60
元 -> 跳到「可買所有」
63.         end
64.
65.         // 情況 10：目前餘額 20 元
66.         S_CAN_20:begin
67.             if(buy_20)    next_state=S_START;    // 買 20 元商品 -> 售出，
餘額清空回起點
68.         end
69.
70.         // 情況 11：目前餘額 50 元以上
71.         S_CAN_ALL:begin
72.             if(buy_20||buy_50)next_state=S_START;// 按下任一商品 -> 售出，餘
額清空回起點
73.         end
74.
75.         // 防止非預期狀態
76.         default:next_state=S_START;// 防止未知狀態，強迫回起點
77.     endcase
78. end
79.
80.     //
=====
=
81.     // 【Sequential Output Logic】
82.     // 在 posedge clk 或 posedge reset 時決定出貨與找零。
83.     //
=====
=
84.     always@(posedge clk or posedge reset) begin
85.         if(reset) begin
86.             // 當 reset 為 1 時，非同步強制關閉所有商品輸出與找零動作

```

```

87.         item_20<= 1'b0;
88.         item_50<= 1'b0;
89.         change  <= 1'b0;
90.
91.     endelse begin
92.         // 清零：將輸出初始化
93.         item_20<= 1'b0;
94.         item_50<= 1'b0;
95.         change  <= 1'b0;
96.
97.         // 判定 1：若累計達 20 元且按下購買 20 元商品
98.         if(current_state==S_CAN_20 &&buy_20)begin
99.             item_20<= 1'b1;      // 給出 20 元商品，不需找零
100.
101.         // 判定 2：若累計達 50 元以上
102.         elseif(current_state==S_CAN_ALL)begin
103.             if(buy_50)begin
104.                 item_50<= 1'b1;    // 選擇 50 元商品 -> 給出 50 元商品
105.             elseif(buy_20)begin
106.                 item_20<= 1'b1;    // 選擇 20 元商品 -> 給出 20 元商品
107.                 change  <= 1'b1;  // 存了 50 元以上卻買 20 元，同步拉高「找零錢」
108.             end
109.         end
110.     end
111. end
112.
113. Endmodule
114.

```

testbench：

```

1.  `timescale 1ns/100ps    // 設定時間單位為 1ns，時間精度為 100ps(例如 #10 代表
    延遲 10ns)
2.  `include "VendingMachine.v" // 導入被測元件 (UUT) 的設計檔案
3.
4.  module vending_machine_tb;
5.      // 宣告佈線，連接 UUT 的暫存器與線路
6.      reg clk;
7.      reg reset;
8.      reg coin_10;

```

```

9.     reg coin_50;
10.    reg buy_20;
11.    reg buy_50;
12.
13.    // 宣告 wire , 觀測晶片輸出
14.    wire item_20;
15.    wire item_50;
16.    wire change;
17.
18.    //
=====
=
19.    // 實例化自動販賣機 (Instantiate the Module)
20.    //
=====
=
21.    vending_machine uut (
22.        .clk(clk),
23.        .reset(reset),
24.        .coin_10(coin_10),
25.        .coin_50(coin_50),
26.        .buy_20(buy_20),
27.        .buy_50(buy_50),
28.        .item_20(item_20),
29.        .item_50(item_50),
30.        .change(change)
31.    );
32.
33.    //
=====
=
34.    // ClockGenerator
35.    //
=====
=
36.    // 每隔 10ns 將 clk 反相一次
37.    // 週期為 20ns , 頻率 50MHz
38.    always#10 clk = ~clk;

```

```

39.
40.    //
=====

=
41.    //WaveformDumpSettings
42.    //
=====

=
43.    // 這段區塊負責產生模擬波形檔，提供給 ModelSim 或 GTKWave 等軟體開啟觀
測
44.    initialbegin
45.        $dumpfile("vending_machine.vcd"); // 指定輸出波形檔
46.        $dumpvars(0,vending_machine_tb); // 把 testbench 下所有參數輸出波形
檔
47.    end
48.
49.    //
=====

=
50.    //TestStimulus
51.    //
=====

=
52.    initialbegin
53.        // 初始化所有輸入訊號
54.        clk = 0;
55.        reset = 1; // 開機，拉高重置訊號 (Active-High)
56.        coin_10 = 0;
57.        coin_50 = 0;
58.        buy_20 = 0;
59.        buy_50 = 0;
60.
61.        #40; // 保持重置狀態兩個時脈
62.        reset = 0; // 電路開始運作
63.        #10; // 等待 10ns 讓電路穩定
64.
65.        //
=====

```

```

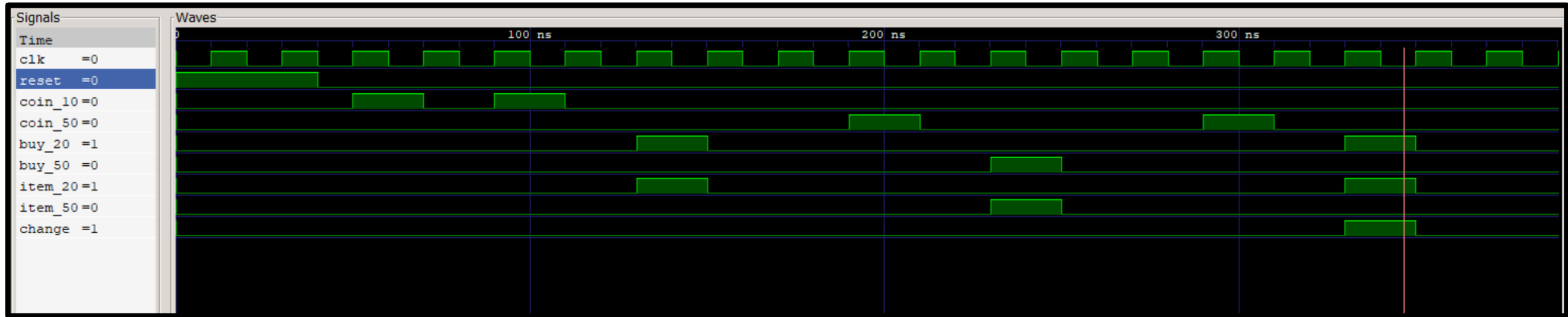
66.      // 【情境 1】：連續投入兩個 10 元硬幣，購買 20 元商品（不找零）
67.      //
=====
68.      $display("時序 [%t]: 開始情境 1(投 10+10 買 20)", $time);
69.
70.      // 投入第一個 10 元
71.      coin_10=1;#20;// 按住 10 元投幣鍵 20ns
72.      coin_10=0;#20;// 放開投幣鍵，狀態機停在 S_LESS 狀態
73.
74.      // 投入第二個 10 元
75.      coin_10=1;#20;
76.      coin_10=0;#20;// 狀態機已跳轉至 S_CAN_20
77.
78.      // 按下購買 20 元商品
79.      buy_20=1; #20;// 按下買 20 元按鈕 20ns (item_20 將會被拉高)
80.      buy_20=0; #40;
81.
82.      //
=====
83.      // 【情境 2】：直接投入 50 元，購買 50 元商品（不找零）
84.      //
=====
85.      $display("時序 [%t]: 開始情境 2(投 50 買 50)", $time);
86.
87.      // 投入 50 元
88.      coin_50=1;#20;
89.      coin_50=0;#20;// 狀態機跳轉至 S_CAN_ALL
90.
91.      // 按下購買 50 元商品
92.      buy_50=1; #20;
93.      buy_50=0; #40;
94.
95.      //
=====
96.      // 【情境 3】：直接投入 50 元，購買 20 元商品（預期找零）
97.      //
=====
98.      $display("時序 [%t]: 開始情境 3(投 50 買 20-> 預期找零)", $time);

```

```
99.  
100.    // 投入 50 元  
101.    coin_50=1;#20;  
102.    coin_50=0;#20;// 狀態機再度進入 S_CAN_ALL  
103.  
104.    // 按下購買 20 元商品  
105.    buy_20=1; #20;// 按下買 20 元按鈕 (item_20 與 change 同步拉高)  
106.    buy_20=0; #40;  
107.  
108.    //--- 結束模擬 ---  
109.    $display("時序 [%t]: 模擬測試結束",$time);  
110.    $finish;  
111. end  
112.  
113. Endmodule  
114.
```

Simulation Result：

```
[Running] cd "d:\lab\VSCode\iVerilog\" && iverilog -o VendingMachine.out VendingMachine.v && vvp VendingMachine.out  
  
[Done] exited with code=0 in 0.098 seconds  
  
[Running] cd "d:\lab\VSCode\iVerilog\" && iverilog -o VendingMachine_tb.out VendingMachine_tb.v && vvp VendingMachine_tb.out  
VCD info: dumpfile vending_machine.vcd opened for output.  
時序 [          500] : 開始情境 1 (投 10+10 買 20)  
時序 [         1900] : 開始情境 2 (投 50 買 50)  
時序 [         2900] : 開始情境 3 (投 50 買 20 -> 預期找零)  
時序 [         3900] : 模擬測試結束  
VendingMachine_tb.v:110: $finish called at 3900 (100ps)  
  
[Done] exited with code=0 in 0.103 seconds
```



說明：

設計核心

「三段式狀態機」架構，將「狀態暫存（Sequential）」與「次態/輸出邏輯（Combinational）」明確分離。

定義了 4 個狀態，分別對應目前的累積金額：初始（0 元）、錢不足（10 元）、可買 20 元、可買所有（50 元以上）。

米利狀態機（Mealy Machine），其次態與輸出不僅取決於「當下狀態」，還會即

時受到「外部輸入訊號（如投幣）」的影響。軟體轉硬體思維的架構，解決了因訊號與時脈邊緣重合所引發的競爭危害（Race Condition），提升了電路穩定度。

程式邏輯概念：

定義狀態（State Encoding）：設計中使用了 2-bit 的暫存器來記錄 4 種累積金額的狀態：S_START (0 元)、S_LESS (10 元)、S_CAN_20 (20 元)、S_CAN_ALL (50 元以上)。

第一段：狀態更新暫存器（Sequential Logic）

只負責在時脈正緣（posedge clk）或重置時，將「次態（next_state）」的數值賦予給「現態（current_state）」。這是整個系統記憶的核心。

第二段：次態組合邏輯（Combinational Logic）

轉態不僅看「現在的狀態」，還必須即時參考「外部輸入」。

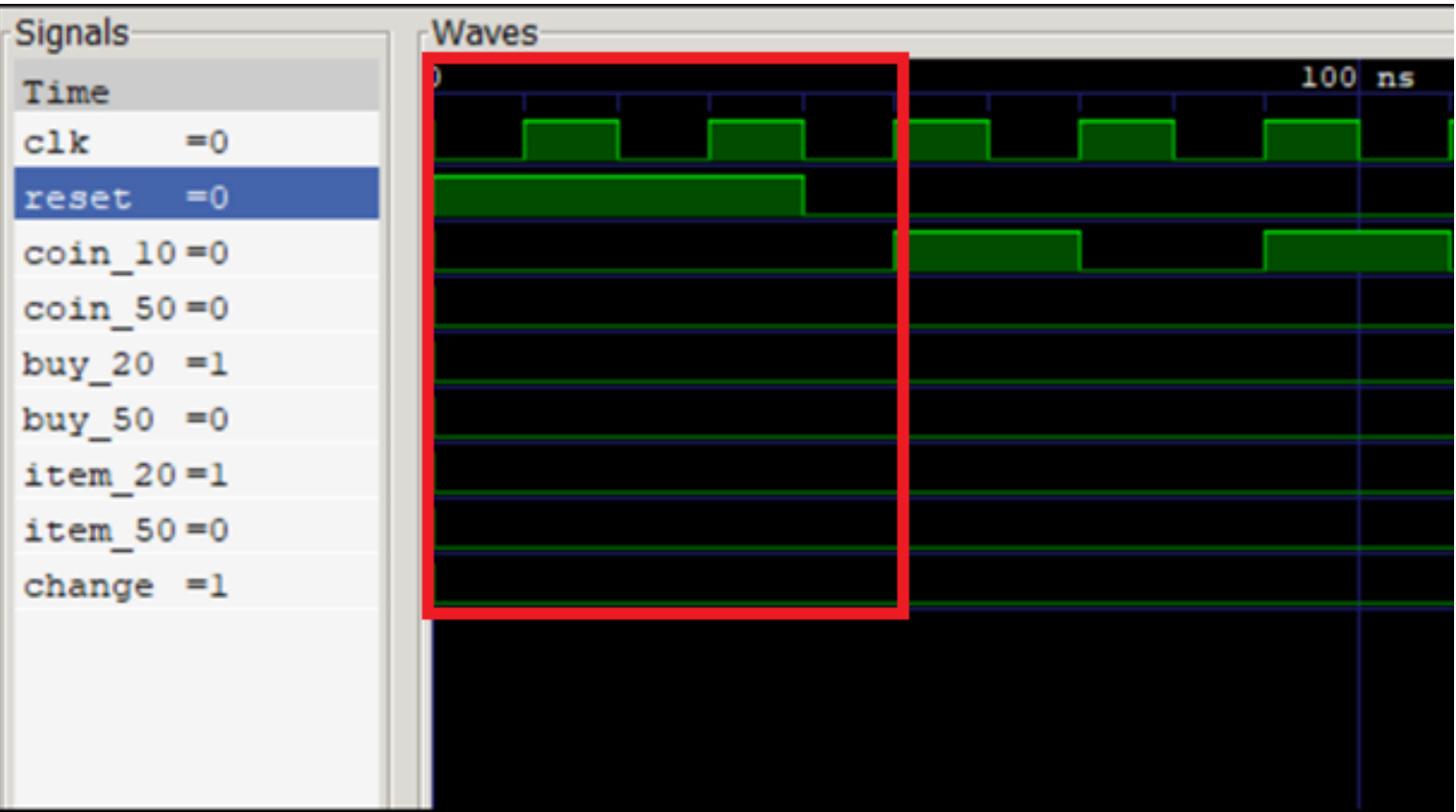
例如：在 S_LESS (10 元) 狀態下，如果輸入 coin_10，次態會變成 S_CAN_20；如果輸入 coin_50，次態則會直接跳到 S_CAN_ALL（累計 60 元，等同可買所有商品）。這個區塊對輸入訊號的反應是即時的。

第三段：輸出邏輯（Sequential Output）

避免組合邏輯產生毛刺(Glitch)導致販賣機「誤按吐出商品」，將輸出邏輯寫在 posedge clk 的循序邏輯區塊中。

只有在時脈觸發的當下，系統確認了「現態」與「購買按鈕（輸入）」的條件，才會拉高 item_20、item_50 或 change 的訊號。

波型時序與情境

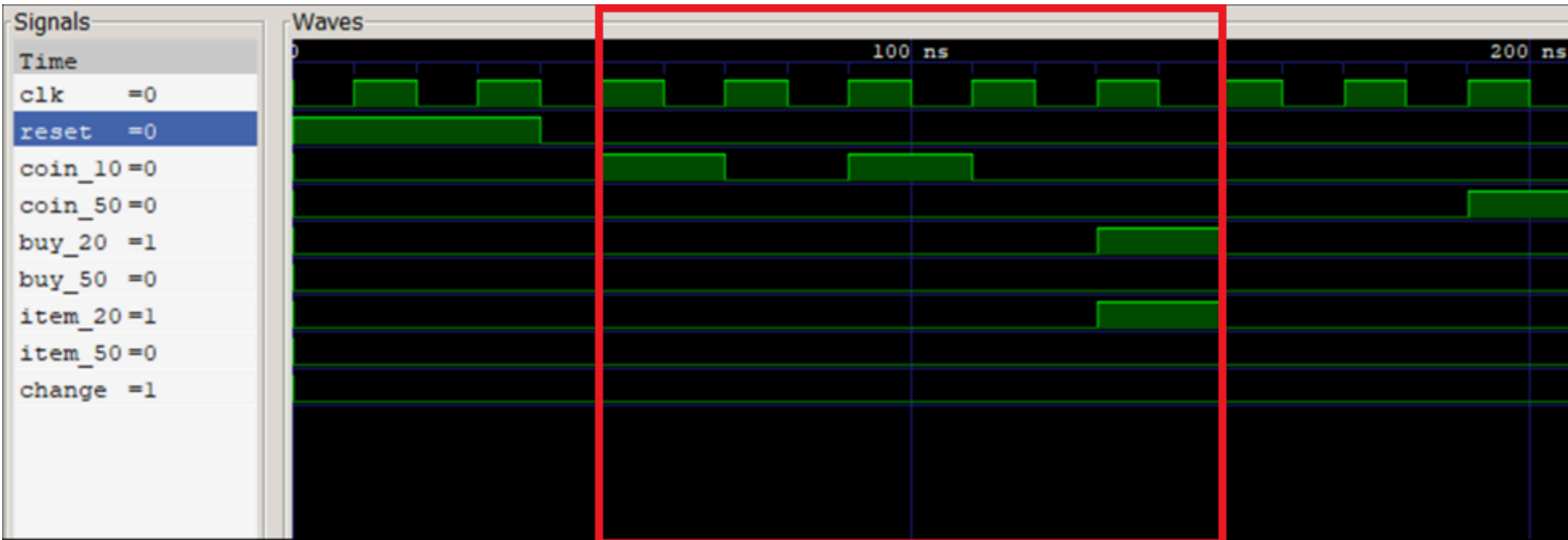


階段 0：系統初始化 (0ns ~ 50ns)

波型表現：在最左側（0ns 到約 40ns），reset 訊號維持在高電位 1。

意義：這段期間強迫狀態機回到初始狀態（餘額 0 元），並且強制關閉所有輸出（item_20、item_50、change 皆為 0），確保販賣機開機時不會亂

吐商品。



階段 1：情境 1 (投 10+10 買 20，不找零)

觸發時間：50ns 開始。

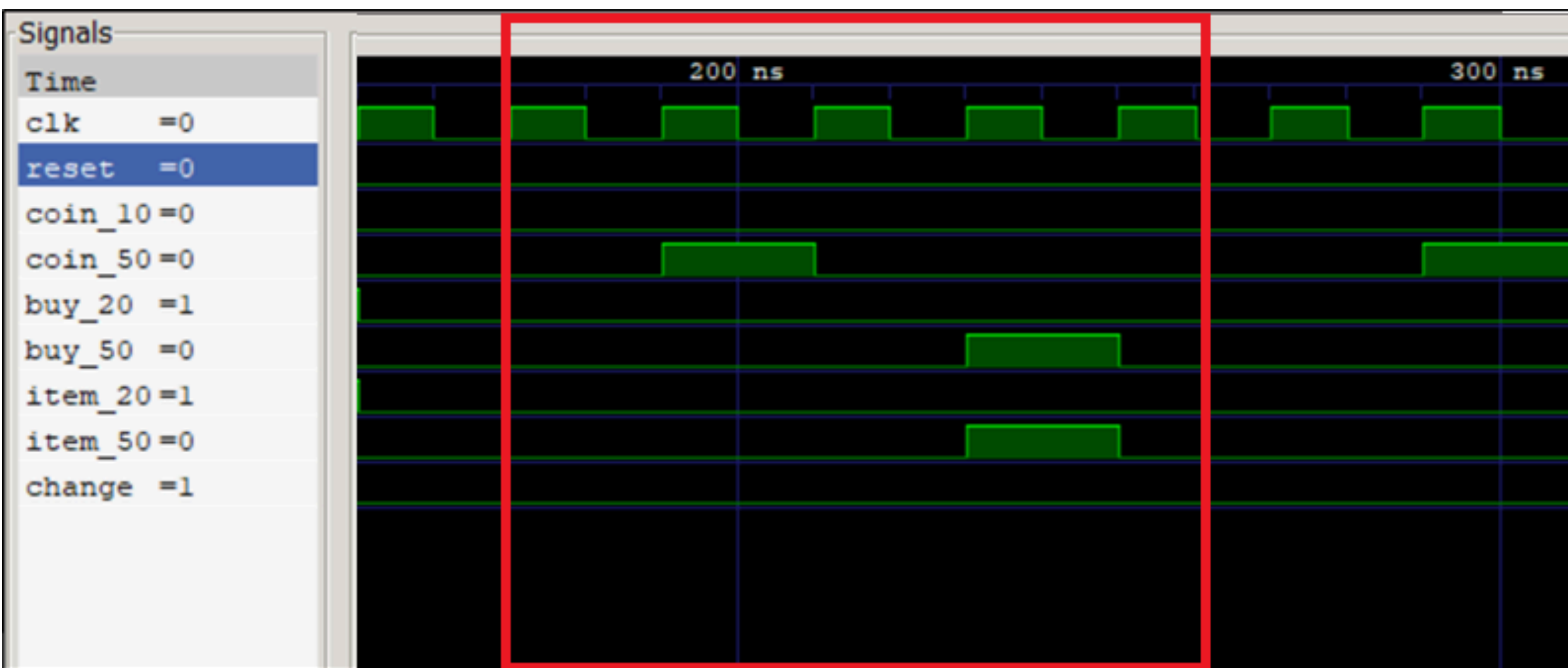
波型表現：

coin_10 在 50ns 左右拉高一次，接著在 90ns 左右再拉高一次（代表連續投幣兩次，狀態機內部累計達 20 元）。

buy_20 隨後在 130ns 左右被按下（拉高）。

輸出觀察：緊接著在下一個時脈正緣 item_20 訊號成功被拉高，而 change 保持為 0。

意義：「金額剛好」時，只出貨、不找零的正確邏輯。



階段 2：情境 2 (直接投 50 買 50，不找零)

觸發時間：190ns 開始。

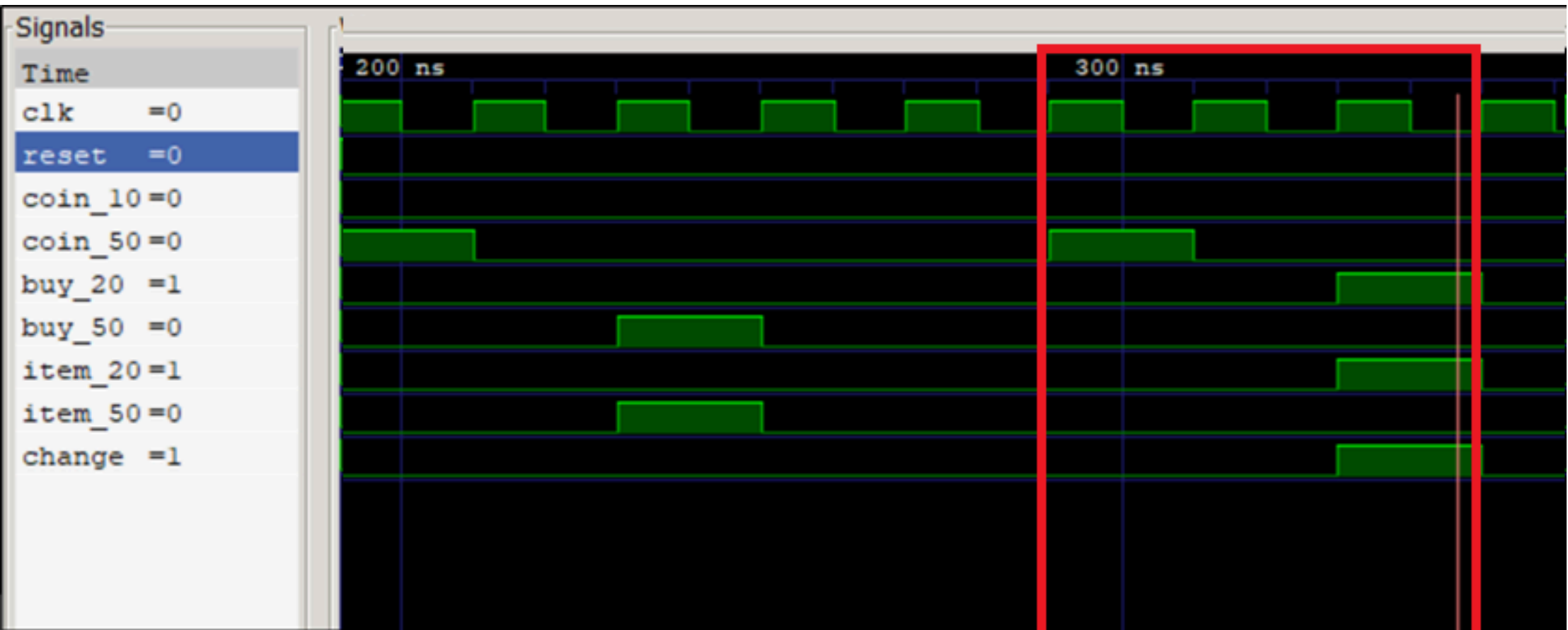
波型表現：

coin_50 拉高一次（狀態機內部直接跳至累計 50 元以上的狀態）。

buy_50 隨後被按下。

輸出觀察：item_50 訊號隨之拉高，其餘輸出維持為 0。

意義：驗證了大面額投幣與對應商品的直接購買邏輯。



階段 3：情境 3 (投 50 買 20，預期找零)

觸發時間：290ns 開始。

波型表現：

coin_50 拉高一次（系統再次擁有了 50 元的額度）。

這一次，使用者按下的是 buy_20。

輸出觀察：在波型圖約 330ns 處，您可以非常清晰地看到，item_20 與 change 這兩條綠色波型在同一個瞬間被「同步拉高」。

意義：米利狀態機（Mealy FSM）結合輸出邏輯的威力。系統在同一個時脈週期內，完成了「給予 20 元商品」的動作，又同時觸發了「退還 30 元零錢」的機構。沒有發生延遲，也沒有因為狀態卡住而少給零錢。

米利狀態機（Mealy FSM）與摩爾狀態機（Moore Machine）應用比較：

	摩爾狀態機 (Moore)	米利狀態機 (Mealy)
反應速度	慢一拍（需等 Clock 觸發轉態）	即時反應（同一個 Clock 週期內完成）
毛刺 (Glitch)	極低（輸出只與暫存器狀態綁定）	高（非同步的輸入變化會直接穿透到輸出）
所需狀態數量	較多（不同輸出必須拆分成獨立狀態）	較少（相同狀態下，可依不同輸入給出不同輸出）

	摩爾狀態機 (Moore)	米利狀態機 (Mealy)
硬體資源	較大 (需較多暫存器來記錄狀態)	較小 (邏輯較為精簡)
系統設計難度	簡單、時序容易收斂 (Timing Closure)	複雜 (需控制組合邏輯的延遲，避免關鍵路徑過長)

摩爾狀態機 (Moore)：輸出訊號乾淨

CPU 指令管線的主控制單元 (MainControlUnit)： 在處理器內部，控制 ALU 或記憶體讀寫的訊號必須乾淨且同步。如果出現一點毛刺，可能會導致錯誤的記憶體覆寫。

交通號誌控制 / 工業電梯控制： 對安全要求極高、且不需要「奈秒級」即時反應的系統。狀態非常明確（紅燈、綠燈），絕不允許因為感測器的一點雜訊而讓號誌亂閃。

SRAM/DRAM 記憶體控制器： 記憶體的讀取/寫入時序 (Read/Write Timing) 非常嚴格，必須按照 Clock 節拍精準給出控制訊號，MooreFSM 能提供最穩定的時序邊界。

米利狀態機 (Mealy)：資源受限、需要極低延遲的邊緣與通訊系統

高速通訊協定的封包解析 (PacketParsing)：在網路交換器或 PCIe 控制器中，封包表頭進來的當下，系統必須在同一個週期決定丟棄還是放行。

匯流排仲裁器 (BusArbiter)： 當多個運算核心同時搶奪同一條資料匯流排時，仲裁器必須在收到 Request 的當下，立刻算出優先權並給出 Grant 訊號，以最大化頻寬利用率。

基礎密碼學 / 序列偵測器 (SequenceDetector)： 在接收連續位元流時（例如尋找封包開頭的 1011 同步碼），MealyFSM 可以用比 Moore 少將近一半的狀態節點完成偵測，大幅節省 FPGA 或 ASIC 的面積。

Homework Q3 : 8-Bit ALU (Behavioral Model)

Verilog HDL :

Icarus Verilog & GTK wave (vscode) :

```
1.  //=====
    =====
2.  //Homework3 : 8-Bit ALU ( Behavioral Model )
3.  //=====
    =====
4.
5.  module ALU8bit( Opcode, Operand1, Operand2, Cin, Result, flagC, flagZ);
6.      // -----
7.      // Ports Declaration
8.      // -----
9.      input [3:0] Opcode;          // 4-bit Opcode (13 種功能需用 4bits)
10.     input [7:0] Operand1, Operand2;    // 8-bit Operand
11.     input Cin;                      // 1-bit 進位/借位輸入 (用於加法與減法)
12.
13.     output reg [15:0] Result = 16'h0000; // 16-bit 結果輸出
14.     output reg flagC = 1'b0,           // Carry flag , 預設 0
15.           flagZ = 1'b0;                // Zero flag) , 預設 0
16.
17.     // -----
18.     // Parameters for Opcodes
19.     // -----
20.     // 運算功能
21.     parameter [3:0] ADD = 4'b0000, // 1. Addition
22.           ADDC = 4'b0001, // 2. Addition with carry
23.           SUB = 4'b0010, // 3. Subtraction
24.           SUBB = 4'b0011, // 4. Subtraction with borrow
25.           DEC = 4'b0100, // 5. Decrement (遞減)
26.           INC = 4'b0101, // 6. Increment (遞增)
27.           TRA = 4'b0110, // 7. Transfer function (輸出等於輸入 , 傳遞)
28.           AND = 4'b0111, // 8. Logical AND
29.           OR = 4'b1000, // 9. Logical OR
30.           XOR = 4'b1001, // 10. Logical XOR
31.           NOT = 4'b1010, // 11. Logical NOT
32.           SHL = 4'b1011, // 12. Left shift (左移 1 位)
```

```

33.         SHR = 4'b1100; // 13. Right shift (右移 1 位)
34.
35.     // -----
36.     // Combinational Logic
37.     // -----
38.     always@ (Opcode or Operand1 or Operand2 or Cin)
39.     begin
40.         // ===== 【Arithmetic Unit】 =====
41.
42.         // 1. Addition
43.         if (Opcode == ADD) begin
44.             Result = Operand1 + Operand2;
45.             flagC = Result[8]; // 以結果的第 8 位元(Bit 8)作為進位溢位判定
46.             flagZ = (Result == 16'h0000); // 零旗標：當 16-bit 結果全為 0 時拉高
47.         end
48.         // 2. Addition with carry
49.         elseif (Opcode == ADDC) begin
50.             Result = Operand1 + Operand2 + Cin;
51.             flagC = Result[8];
52.             flagZ = (Result == 16'h0000);
53.         end
54.         // 3. Subtraction
55.         elseif (Opcode == SUB) begin
56.             Result = Operand1 - Operand2;
57.             flagC = Result[8]; // 減法借位旗標
58.             flagZ = (Result == 16'h0000);
59.         end
60.         // 4. Subtraction with borrow
61.         elseif (Opcode == SUBB) begin
62.             Result = Operand1 - Operand2 - Cin;
63.             flagC = Result[8];
64.             flagZ = (Result == 16'h0000);
65.         end
66.         // 5. Decrement
67.         elseif (Opcode == DEC) begin
68.             Result = Operand1 - 1'b1;
69.             flagC = Result[8];
70.             flagZ = (Result == 16'h0000);

```

```

71.     end
72.     // 6.Increment
73.     elseif (Opcode == INC) begin
74.         Result = Operand1 + 1'b1;
75.         flagC = Result[8];
76.         flagZ = (Result == 16'h0000);
77.     end
78.     // 7.Transfer A
79.     elseif (Opcode == TRA) begin
80.         Result = Operand1;           // 直接將運算元 1 傳送到輸出
81.         flagC = 1'b0;                // 轉移無進位，清除 flagC
82.         flagZ = (Result == 16'h0000);
83.     end
84.
85.     // ===== 【Logic Unit】 =====
86.     // 純邏輯運算不影響/不變更 flagC 的數值
87.
88.     // 8. 邏輯 AND
89.     elseif (Opcode == AND) begin
90.         Result = Operand1 & Operand2;
91.         flagZ = (Result == 16'h0000);
92.     end
93.     // 9. 邏輯 OR
94.     elseif (Opcode == OR) begin
95.         Result = Operand1 | Operand2;
96.         flagZ = (Result == 16'h0000);
97.     end
98.     // 10. 邏輯 XOR
99.     elseif (Opcode == XOR) begin
100.        Result = Operand1 ^ Operand2;
101.        flagZ = (Result == 16'h0000);
102.    end
103.    // 11. 邏輯 NOT
104.    elseif (Opcode == NOT) begin
105.        Result = ~Operand1;
106.        flagZ = (Result == 16'h0000);
107.    end
108.

```

```

109.      // ===== 【Shift Unit】 =====
110.
111.      // 12.Left Shift
112.      elseif (Opcode == SHL) begin
113.          Result = Operand1 << 1;
114.          flagC = Operand1[7];          // 左移時，原本的最高位元(Bit 7)會被擠出去，
            移入進位旗標
115.          flagZ = (Result == 16'h0000);
116.      end
117.      // 13.Right Shift
118.      elseif (Opcode == SHR) begin
119.          Result = Operand1 >> 1;
120.          flagC = Operand1[0];          // 右移時，原本的最低位元(Bit 0)會被擠出去，
            移入進位旗標
121.          flagZ = (Result == 16'h0000);
122.      end
123.
124.      // ===== 【Default Case】 =====
125.      else begin
126.          Result = 16'h0000;
127.          flagC = 1'b0;
128.          flagZ = 1'b0;
129.      end
130.  end
131. Endmodule
132.

```

testbench：

```

1.  `timescale 1ns/100ps    // 設定時間單位為 1ns，時間精度為 100ps
2.  `include "ALU.v"        // 導入 ALU 模組檔名
3.
4.  module ALU8bit_tb;
5.
6.      // Inputs
7.      reg [3:0] Opcode;
8.      reg [7:0] Operand1;
9.      reg [7:0] Operand2;
10.     reg Cin;
11.

```

```

12.    // Outputs
13.    wire[15:0] Result;
14.    wire flagC;
15.    wire flagZ;
16.
17.    // Temporary variable (內部計數)
18.    reg [3:0] count = 4'b0000;
19.
20.    // Instantiate the Unit Under Test (UUT)
21.    ALU8bit uut (
22.        .Opcode(Opcode),
23.        .Operand1(Operand1),
24.        .Operand2(Operand2),
25.        .Cin(Cin),
26.        .Result(Result),
27.        .flagC(flagC),
28.        .flagZ(flagZ)
29.    );
30.
31.    initial begin
32.        // Initialize Inputs (初始化)
33.        Opcode = 4'b0000;
34.        Operand1 = 8'h00;
35.        Operand2 = 8'h00;
36.        Cin = 1'b0;
37.
38.        // 等待 100ns 讓全域重置完成
39.        #100;
40.
41.        // Stimulus
42.        Operand1 = 8'hAA;    // 二進位:10101010
43.        Operand2 = 8'h55;    // 二進位:01010101
44.        Cin = 1'b1;          // 測試帶進位加減法時使用
45.
46.        // 利用 for 迴圈自動掃描 13 種 Opcode 運算結果
47.        for(count = 0; count < 13; count = count + 1'b1) begin
48.            Opcode = count;
49.            #20;            // 每隔 20ns 變換一次運算功能

```

```

50.     end
51.
52.     $finish;          // 結束模擬
53.     end
54.
55.     //
=====
=====
56.     //TerminalMonitor
57.     //
=====
=====
58.     initialbegin
59.
60.         $display("-----
        -----");
61.         $display(" 時間(Time)| Opcode | Operand1 (十六/十進位)| Operand2 (十六/十進
        位)| Cin => Result (十六進位)| C  | Z");
62.
63.         $display("-----
        -----");
64.         $monitor("%8tns| %b | %h  (%3d)  | %h  (%3d)  | %b =>
        %h  | %b | %b",
65.         $time,    Opcode,  Operand1,Operand1,  Operand2,Operand2,  Cin,
        Result,      flagC,flagZ);
66.     end
67.     initialbegin
68.         $dumpfile("ALU_wave.vcd");
69.         $dumpvars(0,ALU8bit_tb);
70.     end
71.
72. Endmodule
73.

```

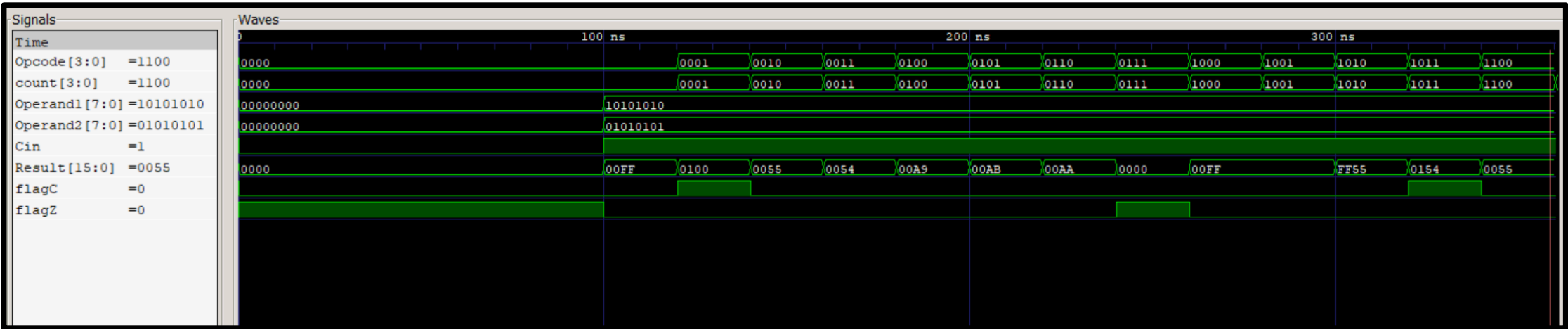
Simulation Result :

```
[Running] cd "d:\lab\VSCode\iVerilog\" && iverilog -o ALU.out ALU.v && vvp ALU.out

[Done] exited with code=0 in 0.103 seconds

[Running] cd "d:\lab\VSCode\iVerilog\" && iverilog -o ALU_tb.out ALU_tb.v && vvp ALU_tb.out
-----
時間(Time) | Opcode | Operand1 (十六/十進位) | Operand2 (十六/十進位) | Cin =>  Result (十六進位) |  C  |  Z  |
-----
VCD info: dumpfile ALU_wave.vcd opened for output.
| 0 ns | 0000 | 00 ( 0) | 00 ( 0) | 0 => 0000 | 0 | 1 |
| 1000 ns | 0000 | aa (170) | 55 ( 85) | 1 => 00ff | 0 | 0 |
| 1200 ns | 0001 | aa (170) | 55 ( 85) | 1 => 0100 | 1 | 0 |
| 1400 ns | 0010 | aa (170) | 55 ( 85) | 1 => 0055 | 0 | 0 |
| 1600 ns | 0011 | aa (170) | 55 ( 85) | 1 => 0054 | 0 | 0 |
| 1800 ns | 0100 | aa (170) | 55 ( 85) | 1 => 00a9 | 0 | 0 |
| 2000 ns | 0101 | aa (170) | 55 ( 85) | 1 => 00ab | 0 | 0 |
| 2200 ns | 0110 | aa (170) | 55 ( 85) | 1 => 00aa | 0 | 0 |
| 2400 ns | 0111 | aa (170) | 55 ( 85) | 1 => 0000 | 0 | 1 |
| 2600 ns | 1000 | aa (170) | 55 ( 85) | 1 => 00ff | 0 | 0 |
| 2800 ns | 1001 | aa (170) | 55 ( 85) | 1 => 00ff | 0 | 0 |
| 3000 ns | 1010 | aa (170) | 55 ( 85) | 1 => ff55 | 0 | 0 |
| 3200 ns | 1011 | aa (170) | 55 ( 85) | 1 => 0154 | 1 | 0 |
| 3400 ns | 1100 | aa (170) | 55 ( 85) | 1 => 0055 | 0 | 0 |
ALU_tb.v:52: $finish called at 3600 (100ps)

[Done] exited with code=0 in 0.099 seconds
```



說明：

設計核心

實作 8-bit 算術邏輯單元 (ALU)，輸入包含 4-bit Opcode、兩組 8-bit Operand 與 Cin，輸出則包含 16-bit Result、flagC 與 flagZ。

共有 13 種功能，包含加法、帶進位加法、減法、帶借位減法、遞減、遞增、Transfer、AND、OR、XOR、NOT、左移與右移，因此 Opcode 由原本常見的 3-bit 擴充為 4-bit。

此 ALU 為純組合邏輯電路，不使用 clock 或狀態暫存器；只要 Opcode 或輸入資料改變，Result 與旗標就會在組合邏輯延遲後更新。

程式邏輯概念

先將 Result、flagC、flagZ、temp 設為預設值，確保每一條路徑都有明確賦值，避免綜合時產生 latch。

算術功能中，ADD 與 ADDC 使用 9-bit temp 保存第 9 位進位；SUB 與 SUBB 則透過 Operand1 是否小於被減數來判斷 borrow，並將結果保留在低 8

位。

邏輯功能包含 AND、OR、XOR、NOT，這些運算不會產生 carry，因此 flagC 維持 0，flagZ 則共用 Result 是否為 0 的判斷。

位移功能中，Left shift 會把 Operand1[7] 作為被移出的 bit 放入 flagC；Right shift 則把 Operand1[0] 放入 flagC。

波型與結果判讀

測試向量固定為 Operand1 = 8'hAA、Operand2 = 8'h55、Cin = 1，Opcode 每 20ns 切換一次，方便在 GTKWave 中逐一觀察 13 種運算結果。

ADD 時 $AA + 55 = FF$ ，因此 Result = 00FF 且 flagC = 0；ADDC 再加上 Cin 後變成 100h，所以 Result = 0100 且 flagC = 1。

AND 時 1010_1010 與 0101_0101 沒有任何同時為 1 的 bit，因此 Result = 0000，flagZ 會被拉高為 1。

Left shift 時 AA 左移後低 8 位為 54，原最高位元為 1，所以 flagC = 1；Right shift 後為 55，原最低位元為 0，所以 flagC = 0。

ALU 電路速度評估

此電路速度主要受組合邏輯傳播延遲影響，路徑大致包含 Opcode 比較、if-else 優先權選擇、算術/邏輯/位移運算，以及最後的 zero flag 比較。

因題目指定使用 if-else，合成器通常會推斷出具有優先權的多工器鏈；越排在後面的功能，如 Left shift、Right shift，選擇路徑可能較長。

實際運算中，加減法需要使用 8-bit 加減法器，延遲通常比 AND、OR、XOR 等單純邏輯閘更高；但在 FPGA 上通常會使用 carry chain，因此仍可維持一定速度。

若未來要提高最高工作頻率，可以改成 case/unique case、平行先算所有結果後再 mux，或將 Arithmetic、Logic、Shift 分組後做階層式選擇，減少單一路徑的邏輯深度。